

Data Programming in Python | BAIS:6040

Homework 6 Solutions. Machine Learning

Instructions

- Fill in the Python statements needed to complete all the questions below.
- Program everything inside this notebook.
- Use only one cell per question.
- Each code cell must be run with no errors.
- Do not contain any unnecessary code in your answer. (Only the final outcome will be regarded as your answer.)
- If the question requests that you store information in a certain variable, use the specific variables name (case sensitive).
- All problems must be solved in a programmable way.
- When you are done, make sure to save your notebook and then click Restart & Run All before submission to create a clean version.

Machine Learning Regression Questions (14 points)

You have a dataframe `dfdi` with the features of diamonds.

```
In [6]: # Do not make any change to this cell. Just run this cell.  
from seaborn import load_dataset  
  
dfdi = load_dataset("diamonds")  
dfdi
```

Out[6]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

53940 rows × 10 columns

For questions 1 to 6, using the diamond data, we aim to build a **regression** model that is able to predict the price of a new diamond based on its carat, cut, color, clarity, depth, table, x, y, and z.

1. There are 3 categorical columns in `dfdi` as shown below: `cut`, `color`, and `clarity`.

In [7]: `dfdi.cut.value_counts().sort_index()`

```
Out[7]: cut
Ideal      21551
Premium    13791
Very Good  12082
Good        4906
Fair        1610
Name: count, dtype: int64
```

In [8]: `dfdi.color.value_counts().sort_index()`

```
Out[8]: color
D      6775
E      9797
F      9542
G     11292
H      8304
I      5422
J      2808
Name: count, dtype: int64
```

In [9]: `dfdi.clarity.value_counts().sort_index()`

```
Out[9]: clarity
IF      1790
VVS1    3655
VVS2    5066
VS1     8171
VS2     12258
SI1     13065
SI2     9194
I1      741
Name: count, dtype: int64
```

Convert those categorical columns into numerical ones.

*** Hint: if you find it hard to use the `apply` method you learned in class because there are too many categorical values to convert, refer to the official solution that uses the `map` method and dictionaries.

```
In [10]: # Your answer here

cut_mapping = {'Fair': 0, 'Good': 1, 'Very Good': 2, 'Premium': 3, 'Ideal': 4}
color_mapping = {'J': 0, 'I': 1, 'H': 2, 'G': 3, 'F': 4, 'E': 5, 'D': 6}
clarity_mapping = {'I1': 0, 'SI2': 1, 'SI1': 2, 'VS2': 3, 'VS1': 4, 'VVS2': 5, 'VVS1': 6}

dfdi['cut'] = dfdi['cut'].apply(lambda x: cut_mapping[x])
dfdi['color'] = dfdi['color'].apply(lambda x: color_mapping[x])
dfdi['clarity'] = dfdi['clarity'].apply(lambda x: clarity_mapping[x])

#Using pd.get_dummies is also acceptable
```

```
In [11]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
dfdi[['cut', 'color', 'clarity']]
```

Out[11]:

	cut	color	clarity
0	4	5	1
1	3	5	2
2	1	5	4
3	3	1	3
4	1	0	1
...	...	...	...
53935	4	6	2
53936	1	6	2
53937	2	6	2
53938	3	2	1
53939	4	6	1

53940 rows × 3 columns

2. According to the goal description above, there are nine features and one target for our prediction model. Based on those features and target, define the two variables `X` as the features set and `y` as the target set.

```
In [12]: # Your answer here
X = dfdi.drop(columns=['price'])
y = dfdi['price']
```

```
In [13]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
X
```

Out[13]:

	carat	cut	color	clarity	depth	table	x	y	z
0	0.23	4	5	1	61.5	55.0	3.95	3.98	2.43
1	0.21	3	5	2	59.8	61.0	3.89	3.84	2.31
2	0.23	1	5	4	56.9	65.0	4.05	4.07	2.31
3	0.29	3	1	3	62.4	58.0	4.20	4.23	2.63
4	0.31	1	0	1	63.3	58.0	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...
53935	0.72	4	6	2	60.8	57.0	5.75	5.76	3.50
53936	0.72	1	6	2	63.1	55.0	5.69	5.75	3.61
53937	0.70	2	6	2	62.8	60.0	5.66	5.68	3.56
53938	0.86	3	2	1	61.0	58.0	6.15	6.12	3.74
53939	0.75	4	6	1	62.2	55.0	5.83	5.87	3.64

53940 rows × 9 columns

In [14]: `# Check your answer here. (Do not make any change to this cell. Just run this cell.`
`y`

Out[14]:

0	326
1	326
2	327
3	334
4	335
...	...
53935	2757
53936	2757
53937	2757
53938	2757
53939	2757

Name: price, Length: 53940, dtype: int64

3. Split `X` and `y` into 80% of training sets `X_train` and `y_train` and 20% of test sets `X_test` and `y_test`. Set `random_state` to 0.

In [15]: `# Your answer here`
`from sklearn.model_selection import train_test_split`
`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)`

In [16]: `# Check your answer here. (Do not make any change to this cell. Just run this cell.`
`X_train.shape, y_train.shape, X_test.shape, y_test.shape`

Out[16]: ((43152, 9), (43152,)), (10788, 9), (10788,))

4. Initialize a linear regression model object `lr` and fit the model using the two training sets.

```
In [17]: # Your answer here
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
Out[17]: LinearRegression
LinearRegression()
```

```
In [18]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
lr
```

```
Out[18]: LinearRegression
LinearRegression()
```

5. Save the training score and test score in the variables `ans5_1` and `ans5_2`, respectively.

```
In [19]: # Your answer here
ans5_1 = lr.score(X_train, y_train)
ans5_2 = lr.score(X_test, y_test)
```

```
In [20]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans5_1, ans5_2
```

```
Out[20]: (0.9063972348424985, 0.9094252854685623)
```

6. Suppose there is a new diamond that has the following features. Using your linear regression model, how much would the price of the new diamond be? Save the predicted price in the variable `ans6`.

```
In [21]: # Do not make any change to this cell. Just run this cell.
d = {"carat": 0.7,
      "cut": 4,
      "color": 1,
      "clarity": 5,
      "depth": 62.4,
      "table": 60.0,
      "x": 5.57,
      "y": 5.61,
      "z": 3.49}

new_diamond = [d["carat"], d["cut"], d["color"], d["clarity"], d["depth"], d["table"]
X_new = [new_diamond]
X_new
```

```
Out[21]: [[0.7, 4, 1, 5, 62.4, 60.0, 5.57, 5.61, 3.49]]
```

```
In [23]: # Your answer here
ans6 = lr.predict(X_new)
```

```
c:\Users\redmo\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X does
not have valid feature names, but LinearRegression was fitted with feature names
warnings.warn(
```

```
In [24]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans6
```

```
Out[24]: array([3226.26842576])
```

Machine Learning Classification Questions (14 points)

```
In [32]: from seaborn import load_dataset
dfc = load_dataset("mpg")
dfc
```

Out[32]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin
0	18.0	8	307.0	130.0	3504	12.0	70	usa
1	15.0	8	350.0	165.0	3693	11.5	70	usa
2	18.0	8	318.0	150.0	3436	11.0	70	usa
3	16.0	8	304.0	150.0	3433	12.0	70	usa
4	17.0	8	302.0	140.0	3449	10.5	70	usa
...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86.0	2790	15.6	82	usa
394	44.0	4	97.0	52.0	2130	24.6	82	europe
395	32.0	4	135.0	84.0	2295	11.6	82	usa
396	28.0	4	120.0	79.0	2625	18.6	82	usa
397	31.0	4	119.0	82.0	2720	19.4	82	usa

398 rows × 9 columns

In [45]: `dfc.dropna(inplace=True)`

1. Change the origin column into 3 separate dummy variable columns ('usa', 'europe', 'japan')

In [46]: `#Answer here`
`dummy_df = pd.get_dummies(dfc['origin'])`
`dfc = pd.concat([dfc, dummy_df], axis=1)`

2. Transform your mpg column into a new categorical variable mpg_level. If it is greater than the median of the column mpg_level = 1, otherwise mpg_level = 0.

```
In [47]: #Answer here
mpg_median = dfc['mpg'].median()
dfc['mpg_level'] = dfc['mpg'].apply(lambda x: 1 if x > mpg_median else 0)
```

```
In [48]: dfc
```

```
Out[48]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin
0	18.0	8	307.0	130.0	3504	12.0	70	usa
1	15.0	8	350.0	165.0	3693	11.5	70	usa
2	18.0	8	318.0	150.0	3436	11.0	70	usa
3	16.0	8	304.0	150.0	3433	12.0	70	usa
4	17.0	8	302.0	140.0	3449	10.5	70	usa
...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86.0	2790	15.6	82	usa
394	44.0	4	97.0	52.0	2130	24.6	82	europe
395	32.0	4	135.0	84.0	2295	11.6	82	usa
396	28.0	4	120.0	79.0	2625	18.6	82	usa
397	31.0	4	119.0	82.0	2720	19.4	82	usa

392 rows × 16 columns



3. Using sklearn, split your dataset into an 80/20 training/testing split

```
In [49]: #Answer here
X = dfc.drop(['mpg', 'name', 'origin', 'mpg_level'], axis=1)
y = dfc['mpg_level']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s
```

4. Perform the following 3 classification techniques using mpg_level as your target (y) variable and all the rest as your predictor (X) variables (do not include name and original mpg in your X variables).

Logistic Regression

KNN with neighbors = 7

Decision Tree

```
In [51]: #Answer here

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

# Logistic Regression
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
log_preds = log_model.predict(X_test)
log_acc = log_model.score(X_test, y_test)

# KNN Classifier
knn_model = KNeighborsClassifier(n_neighbors=7)
knn_model.fit(X_train, y_train)
knn_preds = knn_model.predict(X_test)
knn_acc = knn_model.score(X_test, y_test)

# Decision Tree Classifier
tree_model = DecisionTreeClassifier(random_state=1)
tree_model.fit(X_train, y_train)
tree_preds = tree_model.predict(X_test)
tree_acc = tree_model.score(X_test, y_test)

# Print accuracy scores
print("Logistic Regression Accuracy:", log_acc)
print("KNN Accuracy (k=7):", knn_acc)
print("Decision Tree Accuracy:", tree_acc)
```

Logistic Regression Accuracy: 0.9746835443037974

KNN Accuracy (k=7): 0.8734177215189873

Decision Tree Accuracy: 0.9113924050632911

```
c:\Users\redmo\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

5. Using the training/testing (80/20) split and random_state = 1, evaluate the accuracy of all 3 models and the area under the ROC Curve, sklearn.metrics.roc_auc_score

```
In [54]: #Answer here
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
# Predict probabilities
log_probs = log_model.predict_proba(X_test)[: , 1]
knn_probs = knn_model.predict_proba(X_test)[: , 1]
tree_probs = tree_model.predict_proba(X_test)[: , 1]

# Calculate AUC scores
log_auc = roc_auc_score(y_test, log_probs)
knn_auc = roc_auc_score(y_test, knn_probs)
tree_auc = roc_auc_score(y_test, tree_probs)

print("Logistic Regression ROC_AUC:", log_auc)
print("KNN ROC_AUC (k=7):", knn_auc)
print("Decision Tree ROC_AUC:", tree_auc)
```

Logistic Regression ROC_AUC: 0.9954248366013072

KNN ROC_AUC (k=7): 0.9745098039215687

Decision Tree ROC_AUC: 0.904248366013072

Which model had the highest accuracy? Highest AUC?

Typed out answer here

In both cases it was logistic regression with greater than .97 accuracy and .99 ROC area under the curve.